

Trustiva AI — Product Requirements Document (PRD)

Tagline: *Verify Before You Trust.* **Category:** Cyber Security & AI

Event: HEC · NCEAC · PEC — Generative & Agentic AI Training, Cohort 11 (Midterm Hackathon)

Repository: <https://github.com/Showkat1/trustiva-ai> **Live app:** <https://trustiva-ai.streamlit.app/>

Status: Hackathon MVP **Document version:** 1.0

1. Overview

Trustiva AI is a Generative-AI-powered platform that helps people identify and respond to digital deception before it causes harm. It investigates suspicious digital communication — messages, URLs, screenshots, payment requests, job offers, investment schemes, phishing attempts and impersonation — using a combination of multimodal input handling, generative-AI reasoning, trusted-source retrieval (RAG), deterministic evidence analysis and an explainable risk engine.

Instead of simply labelling an interaction "spam" or "fraud," Trustiva **investigates**: it verifies suspicious claims, correlates multiple risk signals, produces a transparent 0–100 trust/risk score with a plain-language justification, and recommends personalised prevention or recovery actions. The goal is an intelligent digital-trust layer that protects users from financial fraud, identity theft, phishing and social engineering.

2. Problem statement

Digital deception has outpaced the average person's ability to detect it. Fraud now arrives through SMS, WhatsApp, email and social media as fake bank alerts, OTP requests, prize and job scams, investment schemes and government impersonation. Two gaps make this worse:

1. **Detection without explanation.** Existing filters flag content as "spam" or "fraud" without explaining *why*, without verifying the underlying claim, and without telling the victim what to do next.
2. **No localised, evidence-grounded reasoning.** Generic tools miss local context (PKR amounts, Pakistani phone formats, local banks and institutions, Urdu / Roman-Urdu phrasing) and cannot separate an *observation* from *proof*.

Users are left to guess — and guessing is exactly what fraudsters rely on.

3. Goals and non-goals

Goals

- Let a non-expert user investigate any suspicious message, link, screenshot or document in seconds.
- Produce an **explainable** risk assessment, not just a label.
- Ground every conclusion in observable evidence and trusted knowledge.
- Provide clear, risk-tiered protection and recovery guidance.
- Localise reasoning for the Pakistani context.

Non-goals (for the MVP)

- Not a legal or law-enforcement tool; it does **not** make definitive criminal accusations.
- Not a real-time network/endpoint security product.
- Not an authenticated, multi-tenant production service (single-user Streamlit MVP).
- Does not perform live external threat-intel lookups or live URL crawling in the MVP.

4. Target users

- **Everyday consumers** receiving suspicious bank, government or delivery messages.
- **Job seekers and small investors** targeted by fake-offer and investment scams.
- **Less tech-savvy family members** who need a simple "is this safe?" second opinion.
- **Fraud-awareness trainers / helpdesks** who need an explainable teaching tool.

5. Solution overview

A user submits evidence through one of four channels. Trustiva runs it through an eight-stage investigation pipeline and returns a structured investigation report.

```
User Evidence
  → Evidence Extraction
  → URL Intelligence
  → Generative-AI Investigation
  → Trusted-Knowledge Retrieval (RAG)
  → Evidence Correlation
  → Risk Assessment
  → Protection / Recovery Guidance
```

6. Functional requirements (features)

#	Feature	Description
F1	Multimodal intake	Accept a pasted message, a URL, a screenshot (PNG/JPG) or a document (PDF/TXT).
F2	OCR &	Extract text from screenshots via Tesseract OCR and from PDFs via

#	Feature	Description
	document extraction	PyMuPDF.
F3	Structured evidence extraction	Detect URLs, phone numbers, emails, money amounts, organisations, and requested actions (credentials/OTP/payment) via regex NER.
F4	URL intelligence	Deterministically analyse each URL: HTTPS, IP-address host, shorteners, @-obfuscation, punycode, subdomain count, suspicious keywords, and brand/domain mismatch.
F5	Generative-AI investigation	Use an LLM to classify threat category, extract intent/entities/claims, assign risk signals and write an evidence-grounded summary, returned as strict JSON.
F6	Trusted-source RAG	Retrieve the most relevant passages from a curated cyber-fraud knowledge base using semantic search.
F7	Evidence correlation	Combine AI signals, URL findings and extracted evidence into correlated patterns and relationships.
F8	Explainable risk engine	Produce a 0–100 score, a LOW/MEDIUM/HIGH/CRITICAL level, and a plain-language "why this score" explanation.
F9	Protection & response centre	Return risk-tiered, actionable prevention/recovery steps.
F10	Transparent report UI	Show extracted evidence, URL findings, the pipeline, correlations, claims, retrieved sources and the final decision.

7. System architecture

Trustiva is a single Streamlit application that orchestrates a set of independent services.

- **Interface / orchestration:** Streamlit app (`app/main.py`) drives the UI and calls each service in sequence.
- **Intake services:** `OCRService` (screenshots) and a PDF/TXT extractor (documents) normalise all inputs into text.
- **Analysis services:** `EvidenceExtractor` (regex NER) and `URLIntelligence` (deterministic URL parsing) produce observable evidence.
- **Intelligence core:** `llm_service.analyze_message` (generative reasoning) works alongside `RAGService` (semantic retrieval over the knowledge base).
- **Scoring:** `EvidenceCorrelation` links signals; `RiskEngine` (`assess_risk`) computes the explainable score.
- **Response:** `response_service.generate_response` returns tiered protection actions.
- **Data models:** Pydantic models (`InvestigationResult`, `Claim`, `Entity`, `RiskSignal`, `EvidenceReference`, `RiskResult`) keep every stage type-safe.

(See high-level diagram.)

8. Technology stack

- **Frontend / orchestration:** Streamlit + custom CSS
- **Generative AI:** OpenRouter (OpenAI-compatible client), temperature 0, strict-JSON output
- **Retrieval (RAG):** sentence-transformers/all-MiniLM-L6-v2 embeddings + FAISS (IndexFlatL2)
- **Vision & documents:** Tesseract OCR (pytesseract), PyMuPDF (fitz)
- **Evidence analysis:** Python re (regex NER), urllib/validators for URL parsing
- **Configuration:** pydantic-settings (env-based)
- **Testing / evaluation:** pytest unit tests + a dataset-driven evaluation suite
- **Deployment:** Streamlit Community Cloud (packages.txt installs tesseract-ocr)

9. Risk-scoring methodology (explainability)

The risk engine is deliberately **not** a black box. Its design principles:

- **Calibrated weights** for direct harmful actions (credential request, advance payment), pressure tactics (artificial urgency), technical indicators (suspicious URL) and identity/impersonation signals.
- **Correlation bonuses** for dangerous combinations (e.g. credential request + suspicious URL) so the whole is scored higher than the parts — while staying bounded.
- **Anti-double-counting:** contextual classification labels (phishing, job/investment/prize scam, social engineering) receive *low* weights, so a single scam narrative cannot inflate its own score.
- **Critical gating:** reaching CRITICAL requires genuinely independent, corroborating evidence (e.g. known-threat indicator, contradicted claim, or credential+payment+suspicious-URL).
- **Bounded output:** the score is clamped to 0–100.

Risk bands: LOW (0–24) · MEDIUM (25–49) · HIGH (50–79) · CRITICAL (80–100). Every result ships with a "Why this score?" explanation listing the specific indicators found.

10. Knowledge base (RAG corpus)

The MVP ships with a curated, source-attributed cyber-fraud knowledge base:

- Phishing and social-engineering indicators
- Job and investment scam patterns
- FIA cyber-fraud guidance
- SBP digital-payment security guidance

Documents are chunked, embedded and indexed with FAISS. A SHA-256 fingerprint of the knowledge base is stored with the index, so the index rebuilds automatically whenever the corpus changes.

11. User flow

1. User selects an evidence type and provides input.
2. Trustiva normalises input to text (OCR/PDF if needed).
3. Evidence extraction + URL intelligence run.
4. The LLM investigates, using the extracted evidence and retrieved knowledge as context.
5. Signals are correlated; the risk engine scores the case.
6. The report displays evidence, URL findings, correlations, claims, retrieved sources, the risk score with justification, and protection actions.

Primary demo scenario: a fake bank message demanding an OTP and password via a look-alike login link — expected to surface credential-request, suspicious-URL and urgency signals and resolve to a CRITICAL score.

12. Evaluation

A test-case dataset (`data/test_cases/trustiva_test_cases.json`) defines labelled scenarios (phishing, government impersonation, job scam, investment scam, prize scam, ...) with expected risk levels and expected signals. The evaluation suite and unit tests (`tests/`) validate the risk engine, URL service, evidence extraction and correlation behaviour.

13. Safety, ethics & privacy

- Extracted evidence is treated as **observation**, never as proof.
- The system **never** makes a definitive criminal accusation about a person, organisation, number, URL or account.
- An unknown sender is **not** automatically fraudulent; absence of reports is **not** proof of safety.
- Retrieved knowledge is supporting context, not proof about a specific sender.
- The user remains in control — Trustiva provides guidance, not verdicts.
- For production: minimise personal-data collection, add retention controls, authentication, audit logging and jurisdiction-specific official reporting workflows.

14. Success metrics

- **Accuracy:** agreement between predicted and expected risk levels on the test-case set.
- **Explainability:** every result includes a justification and the evidence behind it.
- **Coverage:** the four intake modes and localised evidence patterns all function end-to-end.
- **Usability:** a non-expert can complete an investigation in under a minute.

15. Future roadmap

- Live URL reputation / threat-intelligence integration.
- Separated UI, investigation API, retrieval layer and risk service for scale.

- Authentication, multi-tenant support, audit controls and observability.
- Expanded, continuously-updated knowledge base and multilingual model tuning.
- Official reporting / recovery workflow integrations (e.g. FIA cybercrime reporting).

16. Team

- **Showkat Raja** — Team Lead, Frontend & Backend Engineer
- **Moneka Meghwar** — Documentation Lead
- **Syed Mehtab Ali** — Presentation Lead
- **Ayesha Ashraf** — Research Work
- **Syed Alam Shah Bukhari** — Research Work